

THE SYSTEM AS IT IS BUILT NOW, STEP BY STEP

How Counselor NIDS Is Built

Every part of the system and every step to build it, explained in plain words, with short technical notes for readers who want the details.

Final-year thesis project · October 2026

Before we start: a few words explained

Word	What it means here
Packet	Computers send data in small pieces called packets. A web page arrives as hundreds of them.
Connection (flow)	All the packets of one conversation between two computers. The system judges connections, not single packets.
Intrusion detection system (IDS)	A program that watches traffic and raises an alarm when something looks like an attack. It is the burglar alarm, not the lock: it doesn't block anything.
The AI (machine learning)	Programs that learned what attacks look like from many recorded examples.
Snort	A widely used IDS that recognises known attacks from hand-written rules, like a virus scanner.
Counselor	A detector (or Snort) that another detector asks for advice when it is unsure.
False alarm	The system says "attack" about normal traffic.
Recording (pcap)	A file holding captured network traffic, so it can be replayed and studied.

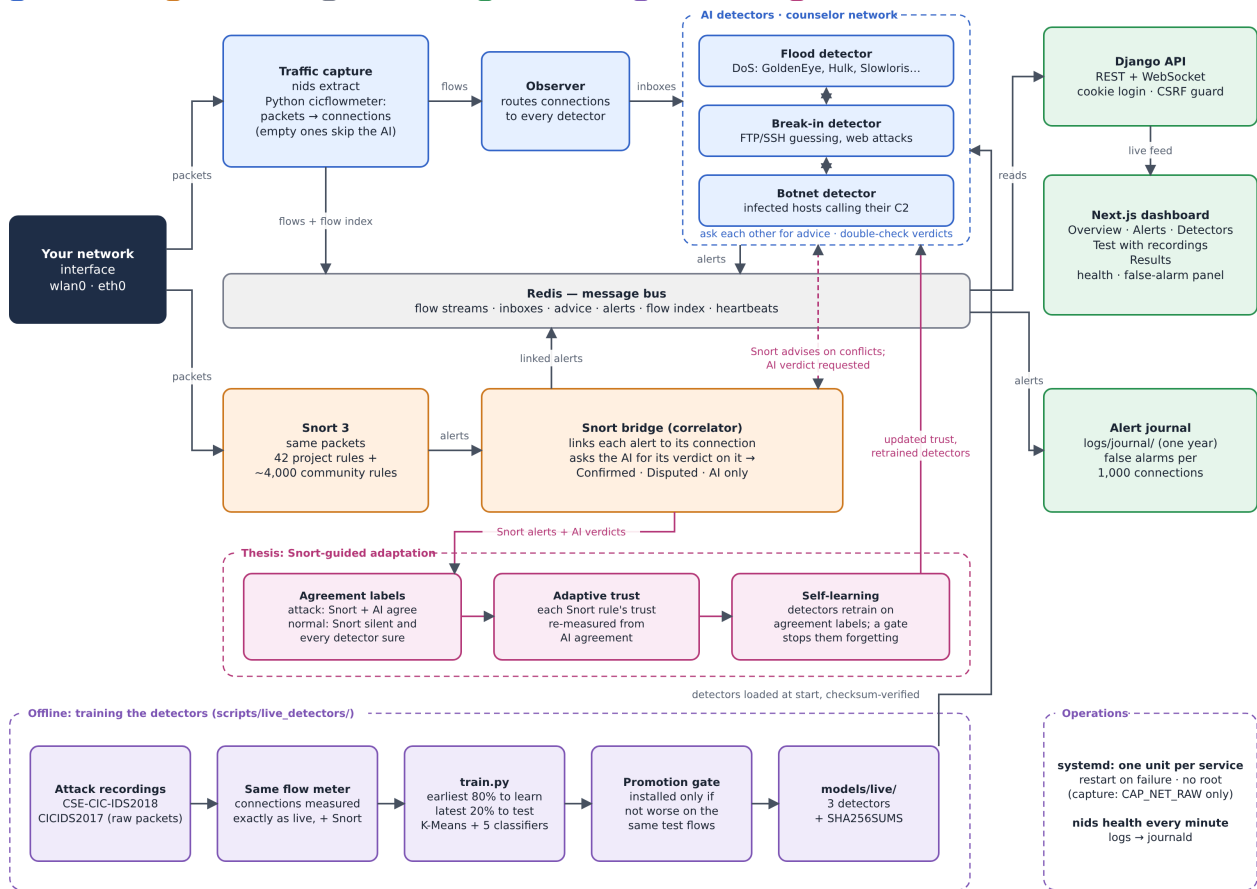
The big picture

Packets from your network go two ways at once. On the **AI side**, they are grouped into connections and judged by three specialist AI detectors that advise each other. On the **Snort side**, Snort checks the same packets against its rules. A **bridge** joins the two: every Snort alarm is matched to its connection and to the AI's opinion of it. A **message board** (Redis) carries everything between the parts. A **web dashboard** shows the result.

Counselor NIDS — system design

Two independent checks watch the same traffic; Snort's alerts are matched to the AI's verdicts, and their agreement teaches the AI on a new network.

■ AI (machine learning)
 ■ Rule checker (Snort)
 ■ Message bus
 ■ API, dashboard, records
 ■ Offline and operations
 ■ Thesis additions



How the parts are connected.

Part A — The parts and how each one works

Step 1 Capturing traffic and turning packets into connections

The **capture** program listens on your network card (for example Wi-Fi, `wlan0`) and gives every packet to a **flow meter**. The flow meter groups packets into connections and measures each one: how long it lasted, how many packets went each way, how big they were, how fast they came. These ~80 measurements are what the AI judges.

- A connection is finished when it closes, has been quiet for 4 minutes, or has run for about 2 minutes.
- Connections that exchanged no data at all (port-scan probes, failed connections) are not given to the AI. It never learned them and would only guess. Snort still sees them.
- Finished connections are sent on every 2 seconds, so even a quiet home network gets answers quickly.
- If the network drops (Wi-Fi disconnects), capture waits and resumes on its own.

TECHNICAL NOTE

`nids extract --live wlan0 ( src/nids/services/live_capture.py )` runs the Python `cicflowmeter` through `src/nids/capture/flowmeter.py`, which fixes two bugs in `cicflowmeter 0.5.0`: its command line passes arguments in the wrong order, and a flow with no forward packets crashed its writer thread. Reading a recording, it ends flows by the recording's own clock, so results are identical on every run.

Step 2 The message board (Redis)

The parts are separate programs. They don't call each other directly; they post to and read from a shared message board. If one part crashes, the others keep going and it can restart and continue.

TECHNICAL NOTE

Redis streams: `nids:unknown` (new connections), `nids:inbox:<detector>`, `nids:alerts`, `nids:snort:alerts`; Redis lists for advice requests and replies; sorted sets for the connection index; a hash of heartbeats for health (`src/nids/services/bus.py`).

Step 3 The Observer hands out the work

The Observer takes each batch of new connections from the board and puts a copy in the inbox of every detector that watches that traffic. It is the post office of the system.

TECHNICAL NOTE

`nids observe ( src/nids/services/observer.py )`: detectors register which traffic sources they subscribe to.

Step 4 Inside one AI detector

Each detector is a specialist. The **Flood detector** knows denial-of-service attacks, the **Break-in detector** knows password guessing and website attacks, and the **Botnet detector** knows infected machines calling their control server. Inside, each one works like this:

1. It puts the connection into a **group of similar connections** it saw during training.
2. For each group it remembers which of its **five AI methods** were best there. Only those vote.
3. If they all agree, that's the verdict: "attack" or "normal".
4. If they disagree, it's a **conflict**, and the detector asks for advice (Step 5).

TECHNICAL NOTE

Groups come from K-Means clustering. The five classifiers are naive Bayes, three decision-tree variants (J48-, REPTree- and RandomTree-like) and a random forest (scikit-learn). Per cluster, the most accurate classifier and any within 0.1% of it are selected. Numbers are log-compressed and scaled first (`src/nids/detector/`). Models: `models/live/*.joblib` .

Step 5 The counselor network: asking for advice

This is the heart of the design. A detector that is unsure asks the others what they think about the same connection. It takes the advice of the most trusted counselor, but only if that counselor is at least 90% accurate. Three rules make it work well on real traffic:

- **Double-check "normal" (cross-check).** A detector that never learned an attack isn't unsure about it; it is confidently wrong. So even a "normal" verdict is checked with the others, and a trusted "attack" wins.
- **No blind guesses.** If the methods disagree and nobody can advise, the connection counts as normal. Guesses caused most false alarms.
- **Snort is a counselor too** (thesis addition, Part D). When a detector is unsure, Snort's opinion counts as advice on connections it alarmed on.

The system's final verdict is "attack" if any detector says attack.

TECHNICAL NOTE

`CounselorNetwork` (`src/nids/counselor/network.py`) with `cross_check_normal=True` and `suppress_fallback=True` . Live, detectors exchange advice over Redis (`RemoteCounselor`); advice is matched to the exact connection.

Step 6 Snort and its rules

Snort reads the same packets and compares them with its rule book: the project's 42 rules plus about 4,000 free community rules. The project's rules cover:

- **website attacks:** SQL injection, cross-site scripting, and password guessing on login pages;
- **password guessing on FTP and SSH**, including slow guessing spread over two minutes;
- **floods**, by counting connection attempts and web requests per second;
- **port scans**, through Snort's scan detector.

Two settings matter:

- **Packets with a "wrong" checksum are not thrown away.** Network cards fill this checksum in at the last moment, so your own computer's outgoing packets look broken to Snort. Dropping them made Snort blind to website attacks.
- **Rules that only report harmless everyday things are switched off**, such as pings and short DNS lifetimes.

TECHNICAL NOTE

`snort/nids.lua` (configuration: `checksum_eval='none'` , `inspectors`, `suppressions`),
`snort/rules/nids.rules` ; community rules in `snort/rules/community/` (downloaded by setup; `NIDS_SNORT_COMMUNITY=0` leaves them out).

Step 7 The bridge: matching Snort alarms to the AI

For every Snort alarm, the bridge finds the connection it belongs to: same two computers, same ports, and a time inside the connection. It then asks the AI detectors their opinion of that connection. The result:

Label	Meaning
Confirmed	Snort and the AI agree it's an attack: very likely real.
Disputed	Only Snort: a Snort false alarm, or an attack the AI never learned.
AI only	Only the AI: typically floods and botnets, which Snort's rules miss.

TECHNICAL NOTE

`nids snort --interface wlan0 ( src/nids/services/snort_bridge.py , flow_index.py )`. A port-scan alarm stands for many probes, so it is matched to every connection between the two computers within ± 30 seconds.

Step 8 The alert journal: counting false alarms over days

Every alarm is also written to a file, one per day, kept for a year. From it the system calculates how many false alarms it raises on your network, per 1,000 connections. When you attack your own machine on purpose to test it, you press "I'm starting an attack test" on the dashboard, so those alarms are not counted as false.

TECHNICAL NOTE

`nids journal` writes `logs/journal/<day>.jsonl`; `nids journal-report` or the dashboard reads it (`src/nids/services/journal.py`).

Step 9 Health checks: is every part running?

Every part sends an "I'm alive" signal about once a second. The dashboard shows a green chip for each part, or a red one if it went quiet for 15 seconds. The command `nids health` gives the same answer in a terminal and can be run every minute automatically.

Step 10 The API: the door to the data

A web server reads the message board and offers the data to the dashboard: counters, alarms, incidents, Snort comparisons, false alarms and health. Live updates arrive every second through a permanent connection.

TECHNICAL NOTE

Django + Django REST Framework + Channels, served by Daphne (`backend/`). Login sets `httpOnly`, `SameSite=Strict` cookies that JavaScript can't read. Changes need a custom header (a CSRF guard). At most 10 login attempts a minute.

Step 11 The dashboard: what you see

Page	What it shows
Overview	What is being watched, part health, a plain-language verdict ("All clear" or "N suspicious connections"), top attackers, alarms over time, the false-alarm panel, latest incidents
Alerts	Every flagged connection with a severity (Critical to Low) and a plain explanation; attackers grouped into attack stories; search; a printable report
Detectors	Each AI detector and how it reached its decisions
Test with recordings	Play a recording of real attacks through the system, without touching live monitoring
Results	How well the detectors did in testing, per kind of attack, with confidence ranges; the whole system on a recording it never saw; and the thesis experiments

It also has alarm notifications, a ready-to-copy firewall command to block an attacker (it never runs anything itself), and offline checks of attacker addresses against public lists of known-bad addresses.

TECHNICAL NOTE

Next.js (React, TypeScript) in `frontend/` ; one WebSocket shared by all pages (`src/lib/live-feed.tsx`).

Step 11b Your verdicts teach the AI

Every alarm has two buttons: **Not an attack** and **Real attack**. Your answer takes the alarm out of the counts straight away. When you have marked a few, **Teach the AI** retrains the detectors on your answers.

A retrained detector is only installed if it gets your marked connections more right *and* hasn't forgotten any kind of attack it knew. The running system then switches to it within a minute, with no restart.

On a real home network this matters: the Flood detector mistook long-lived HTTPS connections for a slow flooding attack. Your "not an attack" teaches it otherwise.

TECHNICAL NOTE

Detectors keep flagged connections' measurements in Redis (`nids:flow:<id>` , 7 days). Verdicts go to `logs/feedback.jsonl` , and `scripts/live_detectors/learn_feedback.py` retrains: each verdict is repeated 20 times, and the gate checks overall detection, every lab cluster and every attack type. Detector services reload a model only if it matches SHA256SUMS .

Part B — How the AI detectors are made

WHY THEY ARE MADE THIS WAY

The AI must learn from connections measured exactly the way live capture measures them. Detectors trained on the public dataset spreadsheets caught 0% of attacks on live traffic, because those spreadsheets were made with a different measuring program. So the detectors are trained from the datasets' original **recordings**, measured by our own flow meter.

Step 12 Getting the recordings

The CSE-CIC-IDS2018 dataset holds recordings of real attacks in a test lab. Its archives are 40–60 GB per day, so the fetch script downloads only the few computers we need (about 1.3 GB). The CICIDS2017 recordings (about 50 GB) add a second lab for the thesis.

TECHNICAL NOTE

`scripts/live_detectors/fetch_captures.py` uses HTTP range requests to pull single files out of remote ZIP archives (`src/nids/capture/remote_zip.py`).

Step 13 Marking which traffic is an attack

For each recording we know which computer attacked which. The script finds the minutes in which the attacker was active by looking at the packets themselves, and marks every connection between attacker and victim in those minutes as that attack. Everything else is normal traffic.

TECHNICAL NOTE

`build_dataset.py` (2018) and `build_cicids2017.py` (2017, labelled from the lab's published schedule and checked against the packets). The 2017 days are split into 32 parts so the slow flow meter can run 4 at a time; a connection is never split across parts.

Step 14 Measuring the connections, and running Snort on them

The marked recordings go through the same flow meter as live traffic. Snort also runs on them, and for every connection we keep which Snort rules fired, if any. The thesis experiment needs that.

TECHNICAL NOTE

Snort alarms are linked to connections offline with exactly the same matching as the live bridge (`src/nids/capture/snort_offline.py`); this was checked to give identical results. Output: `data/processed/live/*.pkl` .

Step 15 Training, and a fair test

For every kind of attack, the earliest 80% of its connections are used to learn and the latest 20% to test. The detectors never see the test traffic while learning, and it comes from later in the attack, as in real life.

TECHNICAL NOTE

`scripts/live_detectors/train.py` . Each detector learns normal traffic plus its own attack family (`KNOWLEDGE`). Sorting is stable, so the same data always gives the same detectors.

Step 16 The safety gate and the seal

New detectors don't replace the working ones automatically. Both sets are tested on the same test traffic, and the new ones are installed only if they are not worse: no more false alarms, and no attack type more than 2 points lower. The installed detector files are then sealed with a fingerprint. If a file is changed, live monitoring refuses to load it, because loading a detector file can run code.

TECHNICAL NOTE

`models/live/manifest.json` records the results; `models/live/SHA256SUMS` holds the fingerprints, checked by `start.sh` and by each systemd detector unit before it starts.

Step 17 Testing the whole system

- **Unseen machine:** a botnet-infected computer the detectors never trained on (`cross_host.py`): 99.8% of its botnet traffic caught.
- **Whole system on a recording:** real attacks the detectors never saw, through the flow meter, the AI and Snort together (`evaluate_capture.py`).
- **On your own network:** the false-alarm journal (Step 8).

Part C — Setting it up and running it

Step 18 One-time setup

`./start.sh setup` does everything needed once:

- creates the Python environment and installs the packages;
- installs the dashboard's packages;
- prepares the database and asks you to create a login;
- downloads Snort's community rules.

The trained detectors already come with the project.

Step 19 Watching your network

`./start.sh live wlan0` starts every part in the right order:

1. Redis;
2. the API and the dashboard;
3. the Observer, the three detectors and the journal;
4. Snort;
5. capture.

It asks for your password once, because reading network traffic needs administrator rights. Open `http://localhost:3000`. Ctrl+C stops everything.

Step 20 Running it permanently, like a product

`sudo deploy/install-systemd.sh wlan0` installs every part as a system service:

- everything starts when the computer boots;
- a part that crashes restarts by itself;
- health is checked every minute;
- nothing runs with full administrator power: capture and Snort get only the permission to read traffic.

TECHNICAL NOTE

One systemd unit per part under `nids.target`, `Restart=on-failure`, `AmbientCapabilities=CAP_NET_RAW CAP_NET_ADMIN`, `NoNewPrivileges`. Detectors are tied to capture (`PartOf`), so a capture restart never mixes two runs. Logs go to the system journal (`journalctl -u 'nids-*`).

Part D — The thesis additions

These parts are built to answer the thesis question: *can the AI adapt on its own to a network it was never trained on, with Snort's help?*

Step 21 Snort becomes a counselor

Before, detectors only asked each other. Now Snort is a counselor as well. When a detector is unsure (its methods disagree), Snort's opinion of that connection counts as advice, and a trusted "attack" advice beats any "normal" one. Otherwise the Flood detector, which knows nothing about website attacks, could confidently overrule Snort on one. Snort only speaks when it raised an alarm. Silence doesn't mean "safe", because Snort cannot see some attacks, such as floods. Snort is not asked to double-check confident "normal" verdicts: tried that way, the system copied every Snort false alarm.

This also runs in the live system whenever Snort is on. A detector that is unsure waits a moment until Snort has seen the whole connection, then asks. On a recording of real attacks, the AI caught 47% of Slowloris this way, against 35% without, with no false alarms.

TECHNICAL NOTE

SnortCounselor (`src/nids/counselor/snort.py`), passed to the network as an *advisor*: a counselor that gives advice but classifies nothing itself. Its advice is worth the trust of the most trusted rule that fired. Live: `nids detect --snort-counselor` reads the rules the bridge linked to each connection from Redis (`src/nids/services/snort_counselor.py`); `NIDS_SNORT_COUNSELOR=0` turns it off.

Step 22 Learning from agreement

On a new network nobody has labelled the traffic. But when Snort and the AI *independently* say "attack", or Snort is silent and every detector is sure a connection is normal, that agreement works as a label. The detectors retrain on these labels. Each detector stays a specialist: it learns the attacks it was involved in, plus normal traffic. A fifth of the labels is kept aside as a quiz, and a retrained detector is only kept if it scores at least as well on the quiz *and* hasn't forgotten anything it knew, checked on the original lab's labelled data. No labels from the new network are needed.

TECHNICAL NOTE

"Independently" means the AI's verdict was not itself caused by Snort's advice, so Snort cannot confirm itself. Retraining puts half of the new labels in the evaluation set, so each group's best methods are re-chosen for the new network.

Step 23 Adaptive trust

How much each Snort rule is trusted used to be fixed. Now it is re-measured on the new network from how often the AI agrees with it. A rule that keeps firing on traffic every detector is sure is normal loses its say: in tests, a noisy Windows policy rule dropped to 0.02 trust, while the real attack rules rose above 0.99. A disagreement only counts when the AI has flagged nothing from that computer. Otherwise a rule would be punished for catching what the AI can't see, which is what happened to the rule that spots Slowloris until this was added. Live, trust is saved to a file, and the Detectors page shows it for every rule.

We also tried re-measuring the AI detectors' own trust the same way. It made things worse, and the reason is a finding of the thesis: agreement labels are the easy cases that every method already gets right, so they make every method look perfect. They can judge Snort's rules, but not the AI's own methods.

TECHNICAL NOTE

Rules: a Beta estimate starting at 0.9, worth 20 observations (`RuleTrust`): $\text{trust} = (0.9 \times 20 + \text{agreements}) \div (20 + \text{agreements} + \text{disagreements})$. The detector variant (`DetectorModel.adapt` , a prior worth 200 samples) is kept for the ablation experiments.

Step 24 The experiment

The detectors are trained on one lab and then watch the other lab's traffic in time order, as if newly installed. After each chunk they are scored on a fixed test set. Each addition is switched on one at a time, and Snort alone and AI-or-Snort serve as baselines. Every rate gets a 95% confidence interval, and McNemar's test says whether each improvement is real or could be chance. The experiment also records how often the agreement labels were really right.

TECHNICAL NOTE

```
python scripts/live_detectors/adapt.py --source 2018 --target 2017 , and the reverse.
```

Results go to `results/adaptation/` . Status: built; it runs once the 2017 recordings are downloaded and processed.

Building it from zero: the checklist

```
# 1. tools: Python 3.11+, Node.js, Redis, Snort 3, tcpdump
# 2. set up and run
./start.sh setup                # environment, packages, database, login, Snort rules
./start.sh live wlan0          # watch the network; dashboard on http://localhost:3000
.venv/bin/nids health          # are all parts running?
.venv/bin/nids journal-report  # false alarms over days

# 3. run it permanently (optional)
sudo deploy/install-systemd.sh wlan0

# 4. rebuild the detectors (optional; they ship ready)
.venv/bin/pip install -e ".[train]"
python scripts/live_detectors/fetch_captures.py    # 2018 recordings (~1.3 GB)
python scripts/live_detectors/build_dataset.py     # mark, measure, add Snort rules
python scripts/live_detectors/build_cicids2017.py  # 2017 recordings (after downloading ~50 GB)
python scripts/live_detectors/train.py             # train; installed only if not worse
python scripts/live_detectors/evaluate_capture.py   # test the whole system

# 5. the thesis experiment
python scripts/live_detectors/adapt.py --source 2018 --target 2017
```

Where each part lives

Part	Folder or file
Capture, flow meter, Snort linking (offline)	src/nids/capture/
Detectors (clusters, methods, training, adaptation)	src/nids/detector/
Counselor network and Snort counselor	src/nids/counselor/
Running services (capture, observer, detector, bridge, journal, health)	src/nids/services/
The <code>nids</code> command	src/nids/cli.py
Snort configuration and rules	snort/
Trained detectors	models/live/
Building, training, testing, thesis experiment	scripts/live_detectors/
API and dashboard	backend/ , frontend/
Running as services	deploy/ , start.sh